

Lesson 13 Structured Note: Infix, Prefix, and Postfix Notation

0. Current Position in the Course

You are now starting **Lesson 13** of the stack curriculum.

Before this lesson, you completed three major phases:

Phase	Lessons	Main Focus
Phase 1	Lessons 1–4	Stack basics, LIFO, array stack, <code>push</code> , <code>pop</code> , <code>peek</code> , helper functions, and display
Phase 2	Lessons 5–8	Linked-list stack, nodes, <code>top</code> pointer, C implementation, and C++ class implementation
Phase 3	Lessons 9–12	Parenthesis and bracket matching using stacks, including <code>()</code> , <code>[]</code> , and <code>{}</code>

So far, you have learned two big things:

1. **How to build stacks** using arrays and linked lists.
2. **How to use stacks** to solve bracket-matching problems.

Now Lesson 13 begins a new phase:

Phase 4: Expression Notation and Conversion

This phase covers:

- Infix notation
- Prefix notation
- Postfix notation
- Operator precedence
- Associativity
- Infix-to-postfix conversion
- Stack-based expression conversion

Lesson 13 is the foundation for this phase.

In simple words:

Before we write programs that convert expressions, we must first understand the different ways expressions can be written.

1. Lesson 13 Goal

By the end of this lesson, you should understand:

- What an expression is.
 - What operands are.
 - What operators are.
 - What infix notation means.
 - What prefix notation means.
 - What postfix notation means.
 - How the same expression can be written in three different forms.
 - Why humans usually use infix notation.
 - Why computers often prefer postfix notation.
 - How precedence affects expression grouping.
 - How brackets affect expression grouping.
 - How same-precedence operators are handled from left to right in this lesson.
 - How to manually convert simple infix expressions into prefix and postfix.
 - How to dry-run conversions step by step.
-

2. What Is an Expression?

An **expression** is a combination of values and operations.

Example:

A + B

This expression means:

Add A and B

An expression usually contains two main parts:

Part	Simple Meaning	Examples
Operand	A value or variable used in the expression	A, B, C, 5, x
Operator	A symbol that performs an operation	+, -, *, /

3. Operand

An **operand** is the value or variable that an operator works on.

Examples of operands:

A
B
C
5
10
x
y

In this expression:

A + B

The operands are:

A and B

Why?

Because **A** and **B** are the values being added.

4. Operator

An **operator** is a symbol that tells us what operation to perform.

Common operators are:

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division

In this expression:

A + B

The operator is:

+

Why?

Because **+** tells us to add **A** and **B**.

5. Expression Example

Expression:

A + B

Breakdown:

Symbol	Role
A	Operand
+	Operator
B	Operand

So the expression has this structure:

Operand Operator Operand

6. Three Ways to Write an Expression

Lesson 13 introduces three expression notations:

1. **Infix**
2. **Prefix**
3. **Postfix**

These three notations can represent the same expression.

The difference is:

The position of the operator changes.

7. Infix Notation

7.1 Meaning of Infix

In **infix notation**, the operator is written **between** the operands.

Structure:

Operand Operator Operand

Example:

A + B

Here:

- A is the first operand.
- + is the operator.
- B is the second operand.

The operator is in the middle, so this is **infix**.

7.2 Why Infix Feels Natural

Infix is the form humans usually write in mathematics.

Examples:

A + B
A - B
A * B
A / B

This form feels natural because we are used to seeing the operator between the values.

For example:

5 + 3

means:

Add 5 and 3

7.3 Infix Examples

Infix Expression	Meaning
A + B	Add A and B
A - B	Subtract B from A
A * B	Multiply A and B
A / B	Divide A by B

8. Prefix Notation

8.1 Meaning of Prefix

In **prefix notation**, the operator is written **before** the operands.

Structure:

Operator Operand Operand

Example:

+ A B

This means the same thing as:

A + B

The operator comes first, so this is called **prefix**.

The word **prefix** means something placed before.

8.2 Prefix Example

Infix:

A + B

Prefix:

+ A B

Explanation:

- The operation is addition.
- The operator **+** is placed before the operands.
- The operands are **A** and **B**.

So:

+ A B

means:

Add A and B

8.3 Prefix Examples

Infix	Prefix
A + B	+ A B
A - B	- A B
A * B	* A B
A / B	/ A B

9. Postfix Notation

9.1 Meaning of Postfix

In **postfix notation**, the operator is written **after** the operands.

Structure:

Operand Operand Operator

Example:

A B +

This means the same thing as:

A + B

The operator comes last, so this is called **postfix**.

The word **postfix** means something placed after.

9.2 Postfix Example

Infix:

A + B

Postfix:

A B +

Explanation:

- The operands **A** and **B** come first.
- The operator **+** comes after them.

So:

A B +

means:

Add A and B

9.3 Postfix Examples

Infix	Postfix
A + B	A B +
A - B	A B -
A * B	A B *
A / B	A B /

10. Same Meaning, Different Arrangement

The same expression can be written in three different ways.

Example:

Notation	Expression
Infix	A + B
Prefix	+ A B
Postfix	A B +

All three mean:

Add A and B

The mathematical meaning does not change.

Only the position of the operator changes.

10.1 Simple Comparison

Infix:	$A + B$	operator is in the middle
Prefix:	$+ A B$	operator is before the operands
Postfix:	$A B +$	operator is after the operands

Important idea:

Prefix and postfix are not new mathematics. They are just different ways of writing the same expression.

11. Why Do We Care About Postfix?

Humans usually prefer infix notation.

For example:

$$A + B * C$$

A human knows that multiplication happens before addition.

So we understand this as:

$$A + (B * C)$$

But a computer must carefully analyze:

- Which operator comes first?
- Which operator has higher priority?
- Are there brackets?
- Are operators of the same priority being used?

Postfix notation makes expression evaluation easier for computers.

11.1 Why Postfix Is Easier for Computers

In postfix notation, the operators appear after the operands they work on.

Example:

Infix:

$A + B * C$

Postfix:

$A B C * +$

This postfix expression clearly says:

First do $B C *$
Then add A to that result

So the order of work becomes easier to process.

Later, postfix expressions can be evaluated using a stack.

That is why postfix is important in a stack course.

12. The BODMAS / Precedence Problem

In infix notation, the order of operations is not always left to right.

Example:

$A + B * C$

A beginner might read from left to right and think:

$A + B$ first
then multiply by C

But that is wrong.

Multiplication has higher precedence than addition.

So the correct grouping is:

$$A + (B * C)$$

This is why precedence matters.

13. Operator Precedence in Lesson 13

Precedence means priority.

Operators with higher precedence are handled first.

For Lesson 13, use this table:

Priority	Operators	Meaning
Highest	()	Brackets are handled first
Medium	*, /	Multiplication and division
Lowest	+, -	Addition and subtraction

13.1 Brackets Have Highest Priority

Example:

$$(A + B) * C$$

Because of brackets, we must handle:

$$A + B$$

first.

So the expression means:

$$(A + B) * C$$

not:

$$A + (B * C)$$

13.2 Multiplication and Division Have Higher Priority Than Addition and Subtraction

Example:

$$A + B * C$$

Here, $*$ has higher precedence than $+$.

So we do:

$$B * C$$

first.

The expression means:

$$A + (B * C)$$

13.3 Addition and Subtraction Have Lower Priority

Example:

$$A * B + C$$

Here, $*$ happens before $+$.

So the expression means:

$$(A * B) + C$$

14. Same Precedence: Left-to-Right Rule

Sometimes operators have the same precedence.

Examples:

$A + B - C$
 $A * B / C$

In Lesson 13, when operators have the same precedence, we handle them from **left to right**.

14.1 Example: $A + B - C$

Both $+$ and $-$ have the same precedence.

So we go from left to right:

$(A + B) - C$

First:

$A + B$

Then:

$\text{result} - C$

14.2 Example: $A * B / C$

Both $*$ and $/$ have the same precedence.

So we go from left to right:

$(A * B) / C$

First:

$A * B$

Then:

result / C

15. Manual Conversion Method

Before using stacks to convert expressions, we first learn manual conversion.

This helps you understand what the final answer should look like.

Use this beginner method:

1. Find the part of the expression that must happen first.
2. Convert that small part into prefix or postfix.
3. Treat that converted part as one unit.
4. Continue until the whole expression is converted.

15.1 Why Use Square Brackets While Learning?

Sometimes we write square brackets like this:

$A + [B * C]$

The square brackets are not part of the real expression.

They are only a learning tool.

They help us remember:

This part has already been grouped or converted.

16. Basic Conversion Rule for Two Operands

For a simple expression:

A + B

The forms are:

Notation	Rule	Result
Infix	Operator between operands	A + B
Prefix	Operator before operands	+ A B
Postfix	Operator after operands	A B +

This basic rule is used again and again in larger expressions.

17. Example 1: Convert A + B * C

17.1 Given Expression

A + B * C

We want to convert it into:

- Prefix
- Postfix

17.2 Step 1: Decide What Happens First

The expression is:

A + B * C

Operators present:

+
*

Multiplication has higher precedence than addition.

So this part happens first:

$B * C$

So we can think of the expression as:

$A + [B * C]$

17.3 Step 2: Convert $B * C$

Infix:

$B * C$

Prefix:

$* B C$

Postfix:

$B C *$

17.4 Step 3: Combine with $A +$

The full expression means:

$A + [B * C]$

Now treat $[B * C]$ as one unit.

17.5 Prefix Conversion

For prefix, the operator comes before the operands.

The main operator is:

+

The two parts are:

A
* B C

So the prefix form is:

+ A * B C

17.6 Postfix Conversion

For postfix, the operator comes after the operands.

The two parts are:

A
B C *

The main operator **+** comes at the end.

So the postfix form is:

A B C * +

17.7 Final Answer for Example 1

Notation	Result
Infix	A + B * C
Prefix	+ A * B C
Postfix	A B C * +

17.8 Meaning of the Postfix Result

Postfix:

A B C * +

This means:

1. First multiply B and C.
2. Then add A to the result.

So it matches:

A + (B * C)

18. Example 2: Convert A + B + C * D

18.1 Given Expression

A + B + C * D

We want to convert it into:

- Prefix
- Postfix

18.2 Step 1: Decide What Happens First

The expression is:

A + B + C * D

Operators present:

+
+
*

Multiplication has higher precedence than addition.

So this part happens first:

$C * D$

Think of the expression as:

$A + B + [C * D]$

18.3 Step 2: Convert $C * D$

Infix:

$C * D$

Prefix:

$* C D$

Postfix:

$C D *$

18.4 Step 3: Handle $A + B$

Now we still have:

$A + B + [C * D]$

The two $+$ operators have the same precedence.

So we handle from left to right.

That means we first handle:

$A + B$

Prefix:

$+ A B$

Postfix:

$A B +$

Now the expression becomes:

$[A + B] + [C * D]$

18.5 Step 4: Combine the Two Parts

The main operation is now addition between these two parts:

$[A + B]$

and

$[C * D]$

18.6 Prefix Conversion

For prefix, the main operator comes first.

Main operator:

+

Left part prefix:

+ A B

Right part prefix:

* C D

So the final prefix expression is:

+ + A B * C D

18.7 Postfix Conversion

For postfix, the main operator comes last.

Left part postfix:

A B +

Right part postfix:

C D *

Main operator:

+

So the final postfix expression is:

A B + C D * +

18.8 Final Answer for Example 2

Notation	Result
Infix	$A + B + C * D$
Prefix	$+ + A B * C D$
Postfix	$A B + C D * +$

19. Example 3: Convert $(A + B) * (C - D)$

19.1 Given Expression

$(A + B) * (C - D)$

We want to convert it into:

- Prefix
- Postfix

19.2 Step 1: Brackets Have Highest Priority

The expression has brackets:

$(A + B)$
 $(C - D)$

Because brackets have the highest priority, we solve inside each bracket first.

19.3 Step 2: Convert $A + B$

Infix:

$A + B$

Prefix:

+ A B

Postfix:

A B +

19.4 Step 3: Convert C - D

Infix:

C - D

Prefix:

- C D

Postfix:

C D -

19.5 Step 4: Multiply Both Results

The full expression is:

[A + B] * [C - D]

The main operator is:

*

19.6 Prefix Conversion

For prefix, the main operator comes first.

Main operator:

*

Left part prefix:

+ A B

Right part prefix:

- C D

So the final prefix expression is:

* + A B - C D

19.7 Postfix Conversion

For postfix, the main operator comes last.

Left part postfix:

A B +

Right part postfix:

C D -

Main operator:

*

So the final postfix expression is:

A B + C D - *

19.8 Final Answer for Example 3

Notation	Result
Infix	$(A + B) * (C - D)$
Prefix	$* + A B - C D$
Postfix	$A B + C D - *$

20. Important Beginner Idea

Do not think of prefix and postfix as completely new expressions.

They are the same expression written in a different arrangement.

For example:

A + B

can be written as:

Infix: A + B
Prefix: + A B
Postfix: A B +

All three mean the same thing.

The only difference is the position of the operator.

21. How to Think During Conversion

When converting an expression, ask yourself these questions:

1. What part must happen first?
2. Are there brackets?
3. Are there multiplication or division operators?
4. Are there addition or subtraction operators?
5. If operators have the same precedence, which one comes first from the left?

6. What is the main operator of the current expression?
7. For prefix, where should the operator go?
8. For postfix, where should the operator go?

22. Prefix vs Postfix: Main Difference

Notation	Where the Operator Goes	Example for <code>A + B</code>
Prefix	Before operands	<code>+ A B</code>
Postfix	After operands	<code>A B +</code>

Simple memory trick:

Prefix -> operator is PRE before operands
Postfix -> operator is POST after operands

23. Why This Lesson Matters for Stacks

This lesson prepares you for stack-based expression conversion.

Later, you will learn how to convert infix to postfix using a stack.

The stack will help manage:

- Operators
- Precedence
- Brackets
- Order of operations

But before using a stack, you must understand what the correct postfix answer should look like.

That is why Lesson 13 focuses on manual conversion.

24. Common Beginner Mistakes

Mistake 1: Thinking prefix and postfix change the meaning

Wrong idea:

$A + B$, $+ A B$, and $A B +$ all mean different things.

Correct idea:

They all mean $A + B$.

Mistake 2: Ignoring precedence

Expression:

$A + B * C$

Wrong grouping:

$(A + B) * C$

Correct grouping:

$A + (B * C)$

Reason:

$*$ has higher precedence than $+$

Mistake 3: Forgetting brackets have highest priority

Expression:

$(A + B) * C$

Correct first step:

$A + B$

Reason:

Brackets have highest priority.

Mistake 4: Confusing prefix and postfix

For:

$A + B$

Correct prefix:

$+ A B$

Correct postfix:

$A B +$

Do not write:

$+ B A$

unless the expression really means:

$B + A$

For subtraction and division, order is very important.

Example:

$A - B$

Prefix:

$- A B$

Postfix:

A B -

This means:

A minus B

not:

B minus A

Mistake 5: Treating same-precedence operators randomly

Expression:

A + B - C

Correct grouping in this lesson:

(A + B) - C

Reason:

+ and - have the same precedence, so we go left to right.

25. Quick Checkpoint Questions

Use these questions to test yourself.

Question	Answer
In <code>A + B</code> , where is the operator?	Between the operands
What notation is <code>A + B</code> ?	Infix

Question	Answer
In $+ A B$, where is the operator?	Before the operands
What notation is $+ A B$?	Prefix
In $A B +$, where is the operator?	After the operands
What notation is $A B +$?	Postfix
In $A + B * C$, what happens first?	$B * C$
Why does $B * C$ happen first?	$*$ has higher precedence than $+$
What is the postfix of $A + B * C$?	$A B C * +$
What is the prefix of $A + B * C$?	$+ A * B C$
What is the prefix of $(A + B) * (C - D)$?	$* + A B - C D$
What is the postfix of $(A + B) * (C - D)$?	$A B + C D - *$

26. Practice Problems

Try converting these expressions into prefix and postfix.

Practice 1

Infix:

$A * B + C$

Think:

$(A * B) + C$

Prefix:

$+ * A B C$

Postfix:

$A B * C +$

Practice 2

Infix:

$A + B * C - D$

First handle multiplication:

$B * C$

Then handle addition and subtraction from left to right:

$(A + (B * C)) - D$

Prefix:

$- + A * B C D$

Postfix:

$A B C * + D -$

Practice 3

Infix:

$(A - B) / C$

Brackets first:

$A - B$

Then divide by `C`.

Prefix:

`/ - A B C`

Postfix:

`A B - C /`

27. Mini Summary

Lesson 13 teaches expression notations.

The three main notations are:

Notation	Operator Position	Example
Infix	Between operands	<code>A + B</code>
Prefix	Before operands	<code>+ A B</code>
Postfix	After operands	<code>A B +</code>

Important rules:

- Brackets are handled first.
- Multiplication and division come before addition and subtraction.
- Operators with the same precedence are handled left to right in this lesson.
- Prefix places the operator before operands.
- Postfix places the operator after operands.
- Postfix is important because it is easier for computers to evaluate using a stack.

28. Final Beginner Takeaway

The most important idea from Lesson 13 is:

Infix, prefix, and postfix can represent the same expression. The difference is only where the operator is placed.

For a small expression:

$A + B$

You should immediately know:

Infix: $A + B$
Prefix: $+ A B$
Postfix: $A B +$

For a larger expression:

$A + B * C$

First decide the grouping:

$A + (B * C)$

Then convert:

Prefix: $+ A * B C$
Postfix: $A B C * +$

This understanding is necessary before learning stack-based infix-to-postfix conversion in later lessons.

29. Lesson 13 Revision Checklist

Before moving to the next lesson, make sure you can do these:

- Explain what an operand is.
- Explain what an operator is.
- Identify infix notation.
- Identify prefix notation.
- Identify postfix notation.
- Convert $A + B$ into prefix and postfix.
- Explain why $A + B * C$ is grouped as $A + (B * C)$.
- Convert $A + B * C$ into prefix.
- Convert $A + B * C$ into postfix.
- Explain why brackets are handled first.

- Convert $(A + B) * (C - D)$ into prefix.
 - Convert $(A + B) * (C - D)$ into postfix.
 - Explain why postfix is useful in stack-based expression evaluation.
-

30. End of Lesson 13

You have now completed the basic idea of:

Infix
Prefix
Postfix

The next lessons will build on this by adding more detailed rules such as associativity and then using stacks to convert infix expressions into postfix form.